



Lab III

# Hexagonal Architecture

Camila Rodríguez Águila - C36624

Diseño de Software - II Semestre, 2025



# ¿Qué es?

- La Arquitectura Hexagonal, también llamada Ports & Adapters, busca crear **componentes de aplicación débilmente acoplados**, fácilmente conectables con su entorno mediante puertos y adaptadores
- **Origen breve**: propuesta por Alistair Cockburn en 2005 para evitar dependencias no deseadas entre capas.



# ¿A qué tipo pertenece?

- Orientado hacia
  - Modularidad
  - Independencia
  - Separación de tareas/preocupaciones
- Arquitecturas Limpias
- Arquitecturas por Capas







# Problema común que resuelve

## Enlace entre Software

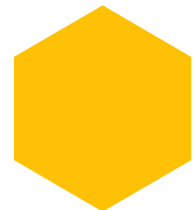
En muchas aplicaciones: la lógica de negocio se enlaza demasiado a frameworks, bases de datos, interfaces de usuario, esto lo hace **difícil de cambiar**

## ¿Qué propone?

:Aislar el núcleo de la lógica de negocio de los detalles de infraestructura, permitiendo intercambiar adaptadores sin cambiar el núcleo.

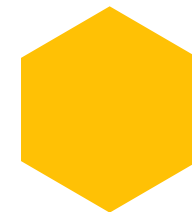
# ¿Cómo mejora el mantenimiento o la escalabilidad del sistema?

## Mantenimiento



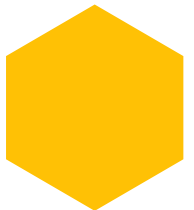
Al separar claramente responsabilidades, los cambios en infraestructura o UI no afectan el núcleo de negocio.

## Testabilidad



El núcleo puede probarse en aislamiento, lo que reduce el coste de pruebas.

## Escalabilidad



Al ser modular se facilita añadir nuevas “adaptadores” (por ejemplo nuevo UI, nuevo DB) sin reescribir la lógica.

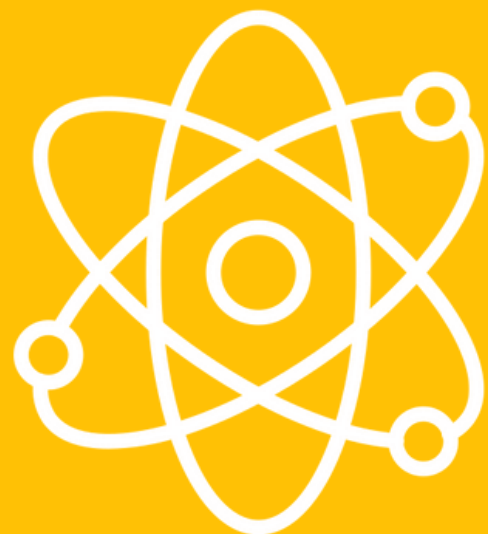
## Flexible



Permite posponer decisiones tecnológicas, cambiarlas cuando sea necesario.



# Partes Principales



## Núcleo de la App

Business Logic



## Puertos

Interfaces



## Adaptadores

Implementaciones externas

# Ventajas y Desventajas

- Acoplamiento débil entre núcleo y detalles.
- Alta testabilidad.
- Flexibilidad para cambiar adaptadores tecnológicas.
- Enfoque en el dominio de negocio (menos atención prematura a infraestructura).

- Mayor complejidad inicial (más interfaces, adaptadores).
- Indirección adicional puede afectar el rendimiento o hacer más difícil el debugging.
- Curva de aprendizaje más pronunciada para desarrolladores no familiarizados.

# Concurrencia

- Aunque la **arquitectura hexagonal** no define cómo manejar concurrencia, facilita su implementación porque el núcleo está aislado de la infraestructura.
- La concurrencia se maneja en los **adaptadores**, por ejemplo mediante colas de mensajes, eventos o threads, mientras el núcleo permanece libre de concurrencia.





# ¿Con qué patrones se suele usar?

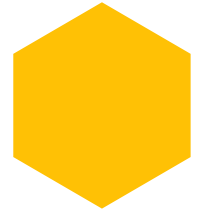
## Domain-Driven Design (DDD)

- Puede combinarse para estructurar el dominio en el núcleo.
- Patrones de diseño: Adaptador (Adapter), Fachada (Facade) – ya que en Hexagonal los “adaptadores” implementan interfaces definidas en puertos.

## Microservicios

- Puede emplearse para diseñar servicios independientes con núcleo desunir de infraestructura.

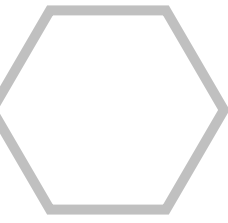
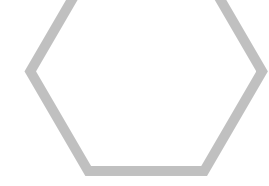
# Datos



El nombre “hexagonal” se refiere a la convención gráfica de representar el núcleo como un hexágono para permitir múltiples puertos, no porque tenga necesariamente seis lados.



Aunque es útil, no siempre necesario: en aplicaciones muy simples quizá su sobre-arquitectura no valga el coste.





# ¡Gracias!

¿Preguntas?

